



Industrial Training Report

on

**“SUMMARIZATION & DECISION TRACIBILITY SYSTEM FOR
SEMICONDUCTOR PRODUCT DEVELOPMENT LIFECYCLE”**

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

MANAS KHANNA

A25305223002

Under the Guidance of

Ms. Anjali Negi

(POST)

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY

Amity University Punjab

Mohali – 140306

2026-27

Certificate

DECLARATION

I, **Manas Khanna**, student of B.Tech in Computer Science and Engineering at Amity University Punjab, hereby declare that the internship work entitled “**Summarization and Decision Traceability System for Semiconductor Product Development Lifecycle**” has been carried out by me during my industrial training at **SCL Mohali** under the guidance of **Mr. Harhspreet Singh**.

This report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering for the academic session 2026–27.

I further declare that this report is based on the software system, documentation corpus, and implementation work present in the project repository studied during the internship period, and that it has not been submitted elsewhere for the award of any other degree or diploma.

Date:

Place:

Manas Khanna

ACKNOWLEDGEMENT

I express my sincere gratitude to my parents for their constant encouragement and support.

I am thankful to **SCL Mohali** for providing me with an opportunity to work in a disciplined technical environment and for allowing me to study a problem at the intersection of semiconductor program management, document processing, and applied Artificial Intelligence.

I am especially grateful to **Mr. Harhspreet Singh** for his guidance, feedback, and support throughout the internship. I also thank the technical teams whose workflows, documentation styles, and traceability needs shaped the direction of the work.

I would also like to thank the faculty of Amity School of Engineering and Technology for their encouragement and academic support during the preparation of this report.

Manas Khanna

ABSTRACT

This internship focused on building a local-first web application for organizing engineering documents produced during a semiconductor product development program and converting those documents into structured, searchable knowledge. The implemented system is titled **Summarization and Decision Traceability System for Semiconductor Product Development Lifecycle**.

The application is built using **FastAPI**, **SQLite**, **SQLAlchemy**, **Jinja templates**, **vanilla JavaScript**, and **Ollama**. It stores projects and documents, extracts raw text from supported inputs, generates summaries, extracts decisions, action items, risks, and timeline events, and presents them through dashboard, search, project, document, and timeline views. The current repository also includes additional code paths for OCR, spreadsheet extraction, image ingestion, and summary translation.

The working material studied in this repository represents a synthetic but realistic semiconductor program: the **Radiation-Hardened Satellite ADC Development Program**. It is designed as a dense engineering traceability corpus with linked technical context, evolving risks, structured metadata, and repeated terminology suitable for search and summarization workflows.

The internship provided practical exposure to software architecture, asynchronous document processing, prompt-driven information extraction, database-backed traceability, frontend integration using the UX4G template style, and the gap between planned functionality and the actual state of a codebase. It also highlighted implementation limits such as dependency drift, configuration mismatches, and the importance of documenting what a system truly does instead of what it was originally intended to do.

Contents

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	vi
List of Figures	vii
List of Tables	vii
1 Company Profile	1
1.1 About SCL Mohali	1
1.2 Relevance to This Internship	1
1.3 Operational Context	1
2 Work Undertaken	1
2.1 Department / Team Allocation	1
2.2 Role Performed	1
2.3 Tools and Technologies Used	2
2.4 Skills Acquired	2
3 Project Title and Domain	3
4 Problem Statement	3
5 Project Objectives	3
6 Corpus Studied	3
6.1 Nature of the Source Material	3
6.2 Program Narrative Observed	4
7 Implemented System Overview	4
7.1 High-Level Architecture	4
7.2 Main Application Boot Flow	5
8 Configuration and Storage	5
8.1 Settings Observed in Code	5
8.2 Important Configuration Mismatch	5
9 Database Design	5
9.1 Purpose of Each Table	6

10 Document Processing Pipeline	6
10.1 Implemented Extraction Logic	6
10.2 Actual Processing Flow	6
10.3 Job Progress Stages	7
11 Ollama Integration	7
11.1 Implemented Service Behavior	7
11.2 Prompting Approach	7
11.3 Fallback Logic	8
12 Background Processing and Synchronization	8
12.1 Two Mechanisms Present in Code	8
12.2 Job Tracking	8
13 User Interface Implemented	8
13.1 Pages Present in the Repository	8
13.2 Frontend Characteristics	9
14 Search and Traceability Features	9
14.1 Search Scope	9
14.2 Available Filters	10
14.3 Highlighting	10
15 Additional Implemented Capabilities	10
16 What the Current Codebase Does Not Fully Solve	10
16.1 Dependency Drift	11
16.2 Documentation Mismatch	11
16.3 Storage Path Design	11
16.4 Search Simplicity	11
16.5 Migration Simplicity	11
16.6 No Authentication Layer	11
17 Learning Outcomes	12
17.1 Technical Learning	12
17.2 Domain Learning	12
17.3 Professional Learning	12
18 Conclusion	13
References	13

List of Figures

List of Tables

1 Company Profile

1.1 About SCL Mohali

Semi-Conductor Laboratory (SCL), Mohali, is a major Indian organization working in semiconductor technology, microelectronics, fabrication, and associated digital systems. The organization operates in a highly process-oriented environment where engineering documentation, reviews, and traceability are critical to decision making.

1.2 Relevance to This Internship

The software problem addressed in this internship is directly relevant to such an environment. Semiconductor programs generate large volumes of technical records, decision history, risk updates, and supporting documentation. Without a structured system, it becomes difficult to reconstruct why a design decision was made, which assumption superseded an earlier one, or how a risk evolved over time.

1.3 Operational Context

The repository studied during the internship models a semiconductor product lifecycle from requirements and architecture to yield learning and production readiness. The software system developed during the internship is meant to support:

- document organization,
- summary generation,
- decision traceability,
- risk visibility,
- project history reconstruction, and
- local search across a technical corpus.

2 Work Undertaken

2.1 Department / Team Allocation

The work aligned most closely with software development, information systems, and technical knowledge-management use cases. The implementation combines backend development, frontend integration, document processing, and AI service orchestration.

2.2 Role Performed

My role during the internship was to study the existing repository, understand the real behavior of the codebase, review the engineering content used by the system, and implement or refine a local-first application capable of turning semiconductor project documents into structured knowledge.

The work included:

- understanding the existing FastAPI application and its database schema,
- reviewing document-processing and Ollama integration code,
- studying the semiconductor program dataset and its traceability structure,
- aligning user-facing pages with a UX4G-inspired layout,
- supporting asynchronous background processing of uploaded files,
- documenting implemented features and current limitations accurately.

2.3 Tools and Technologies Used

- Python
- FastAPI
- SQLAlchemy
- SQLite
- Jinja2
- HTML, CSS, and vanilla JavaScript
- Ollama local API
- Local LLM prompting for summary and extraction
- pdfplumber and python-docx
- Git-based repository workflow
- LaTeX for report preparation

2.4 Skills Acquired

- reading and auditing an existing codebase,
- backend API design,
- database-backed document management,
- asynchronous processing design,
- prompt engineering for structured extraction,
- repository-to-report alignment,
- technical documentation based on real implementation state.

3 Project Title and Domain

Summarization and Decision Traceability System for Semiconductor Product Development Lifecycle

This project is a local-first engineering knowledge system targeted at semiconductor program documentation. Instead of focusing on generic collaboration notes, the implementation centers on design evolution, decision reversals, risk progression, reliability concerns, yield learning, and production-readiness traceability.

4 Problem Statement

Semiconductor product development creates many interdependent technical records over time. These records are often unstructured and distributed across folders, making it difficult to answer questions such as:

- What decision replaced an earlier assumption?
- Which document introduced a particular risk?
- What actions were opened and when were they closed?
- How did the project move from requirements to production readiness?

The need is therefore for a local system that can ingest documents, summarize them, extract structured entities, and preserve a searchable trace of project evolution.

5 Project Objectives

The implemented repository supports the following objectives:

- store semiconductor engineering documents locally,
- create and manage projects,
- extract raw text from supported files,
- generate summaries using a local Ollama model,
- extract decisions, action items, risks, and timeline events,
- present project dashboards and detailed document views,
- support asynchronous ingestion with job-status polling,
- maintain searchable project knowledge without cloud dependence.

6 Corpus Studied

6.1 Nature of the Source Material

The repository contains a synthetic semiconductor program corpus built to simulate a realistic engineering lifecycle from early requirements to production readiness. The source material is intentionally interlinked so that summaries, decisions, risks, action items, and timeline events can be cross-checked against one another.

6.2 Program Narrative Observed

The corpus consistently tracks a semiconductor lifecycle with recurring themes such as:

- ADC Reference Voltage stability,
- Radiation Hardening,
- Lithography Alignment,
- Yield Loss,
- Packaging Revision changes,
- Burn-In Testing,
- Reliability Qualification.

The metadata also captures multiple decision reversals such as:

- D-006 replacing D-003,
- D-007 replacing D-004,
- D-008 replacing pilot screening assumptions tied to D-004,
- D-009 replacing D-005,
- D-010 replacing an earlier ATPG strategy.

7 Implemented System Overview

7.1 High-Level Architecture

The application follows a monolithic but modular local architecture:

- **Frontend:** Jinja-rendered HTML templates with vanilla JavaScript and UX4G-style assets.
- **Backend:** FastAPI routes for pages and JSON APIs.
- **Persistence:** SQLite database managed through SQLAlchemy.
- **AI Layer:** Ollama service for summary and extraction prompts.
- **File Storage:** local filesystem for uploaded and synchronized files.
- **Background Processing:** job records stored in SQLite and processed asynchronously.

7.2 Main Application Boot Flow

The root `main.py` imports the app instance from `app/main.py`. On startup:

- the database schema is initialized,
- lightweight migrations add missing columns,
- the application synchronizes a local document source into the database using `ProjectService.sync_meetings_folder()`.

8 Configuration and Storage

8.1 Settings Observed in Code

The actual runtime configuration is defined in `app/settings.py`. Important observations from the reviewed code are:

- `DATA_DIR` points to `data/`,
- `DATABASE_PATH` points to `data/database/app.db`,
- `UPLOADS_DIR` currently points to a top-level local source directory rather than `data/uploads`,
- the default Ollama model in code is `gemma3:270m`,
- the configured timeout is 25 seconds,
- retries default to 2.

8.2 Important Configuration Mismatch

While the current code uses `gemma3:270m`, the README and `.env.sample` still describe `gemma3:1b`. This discrepancy is important because the repository documentation and the running configuration are not fully aligned.

9 Database Design

The SQLAlchemy models and lightweight migration code create and maintain the following tables:

- `projects`
- `documents`
- `summaries`
- `decisions`
- `action_items`
- `risks`

- `timeline_events`
- `jobs`

9.1 Purpose of Each Table

- `projects`: stores project name, description, and creation timestamp.
- `documents`: stores filename, type, hash, storage path, raw text, and processing status.
- `summaries`: stores generated document summaries.
- `decisions`: stores extracted decision records.
- `action_items`: stores extracted action items.
- `risks`: stores extracted risks.
- `timeline_events`: stores project events for chronological views.
- `jobs`: stores background processing state, stage, progress, and errors.

10 Document Processing Pipeline

10.1 Implemented Extraction Logic

The document processing service currently supports more file types than the original minimal Phase 1 scope. Based on the reviewed code, the extraction logic handles:

- TXT and MD via direct decoding,
- PDF via `pdfplumber` with OCR fallback,
- DOCX via `python-docx`,
- XLSX/XLS via `openpyxl`,
- PNG/JPG/JPEG via OCR.

10.2 Actual Processing Flow

When a document is processed:

1. the file is stored locally,
2. metadata and a content hash are saved,
3. a job record is created,
4. raw text is extracted from disk,
5. existing extracted entities for that document are cleared,

6. a summary is generated,
7. decisions are extracted,
8. actions are extracted,
9. risks are extracted,
10. timeline events are extracted,
11. a text summary file is saved under `data/generated_summaries/`.

10.3 Job Progress Stages

The frontend and backend share the following stage sequence:

- Upload Complete
- Extracting Text
- Generating Summary
- Extracting Decisions
- Extracting Actions
- Extracting Risks
- Updating Timeline

11 Ollama Integration

11.1 Implemented Service Behavior

The `OllamaService` provides explicit methods for:

- `generate_summary()`
- `extract_decisions()`
- `extract_actions()`
- `extract_risks()`
- `extract_timeline()`

11.2 Prompting Approach

The service uses reusable prompts defined in `app/services/prompts.py`. JSON-style prompts are used for entity extraction, while a structured text prompt is used for summaries.

11.3 Fallback Logic

The actual implementation includes a rule-based fallback. If the Ollama request fails or times out, the service generates approximate summaries and JSON extraction outputs using heuristic string matching rather than returning nothing. This means the system can still populate records even when the model is unavailable, although the output quality is lower and less faithful than LLM-generated extraction.

12 Background Processing and Synchronization

12.1 Two Mechanisms Present in Code

The reviewed repository currently uses two asynchronous mechanisms:

- FastAPI `BackgroundTasks` for uploads, edits, replacements, and summary regeneration triggered by API requests.
- An internal queue plus daemon worker thread for startup synchronization of the local `meetings/` directory.

12.2 Job Tracking

Every processing request creates a `jobs` table entry. The frontend polls `/api/jobs/{job_id}` to update status, stage, progress, and error details.

13 User Interface Implemented

13.1 Pages Present in the Repository

From the code and templates reviewed, the application currently provides:

- Dashboard
- Projects page
- Project dashboard
- Upload page
- Document viewer
- Search page
- Timeline page
- Decisions page
- Risks page
- Actions page

13.2 Frontend Characteristics

The frontend is built with:

- Jinja templates,
- vanilla JavaScript,
- local static assets,
- UX4G-inspired layout and stylesheet integration.

The JavaScript implements:

- project creation,
- drag-and-drop file upload,
- click-to-select upload,
- multiple file selection,
- upload progress bars,
- overlay-based processing status,
- tabbed document views,
- live search rendering with highlighted matches.

14 Search and Traceability Features

14.1 Search Scope

The search service currently supports matching against:

- document raw text,
- summaries,
- decisions,
- action items,
- risks,
- timeline events.

14.2 Available Filters

The code supports result-type filtering and project scoping. The result types include:

- all,
- documents,
- decisions,
- actions,
- risks,
- timeline.

14.3 Highlighting

Matched snippets are highlighted using HTML `<mark>` tags generated by the backend before rendering in the search page.

15 Additional Implemented Capabilities

Beyond the core summary-and-traceability workflow, the reviewed code also includes:

- project editing and deletion,
- document moving between projects,
- document editing for text-based files,
- document replacement,
- direct document download,
- summary translation through `/api/documents/{id}/translate`,
- risk severity visualization using emoji traffic lights.

16 What the Current Codebase Does Not Fully Solve

After reading the repository, several gaps between implementation and production-readiness were identified:

16.1 Dependency Drift

The code imports libraries for OCR, image extraction, spreadsheet parsing, and translation such as:

- `pytesseract`,
- `pdf2image`,
- `openpyxl`,
- `PIL`,
- `deep_translator`,
- optional `torch` and `transformers`.

However, these packages are not listed in `requirements.txt`. Therefore, these features are implemented in code but may not run on a fresh environment without manual dependency installation.

16.2 Documentation Mismatch

The README still documents a narrower feature set and a different default model from what the code currently uses.

16.3 Storage Path Design

The code now uses the top-level `meetings/` folder as the upload storage directory, which means runtime files are mixed with repository content rather than isolated cleanly under `data/uploads/`.

16.4 Search Simplicity

Search is implemented using SQL `LIKE/ILIKE`-style matching rather than full-text indexing or semantic retrieval.

16.5 Migration Simplicity

Database migrations are handled by a lightweight “add missing columns” strategy rather than a dedicated migration framework such as Alembic.

16.6 No Authentication Layer

There is currently no authentication, authorization, role control, or audit log around user actions.

17 Learning Outcomes

17.1 Technical Learning

This internship strengthened my understanding of:

- backend application design using FastAPI,
- ORM-driven persistence with SQLAlchemy,
- local-first software design,
- AI-assisted document processing pipelines,
- asynchronous job orchestration,
- frontend integration without large JavaScript frameworks,
- the difference between repository documentation and actual executable behavior.

17.2 Domain Learning

By reading the complete semiconductor program corpus, I also gained better exposure to:

- requirements reviews,
- architecture trade-offs,
- decision reversals,
- risk escalation,
- yield analysis,
- packaging qualification,
- production-readiness tracking.

17.3 Professional Learning

The work also improved my ability to:

- inspect an existing codebase carefully before modifying it,
- document software based on evidence from code and files,
- identify limitations without overstating capabilities,
- connect software features to a real engineering workflow.

18 Conclusion

The internship work reviewed in this repository resulted in a functional local-first engineering traceability application centered on semiconductor program documents. The implemented system is not merely a generic meeting-note summarizer; it is a structured document-processing platform backed by FastAPI, SQLite, and Ollama, with user-facing support for dashboards, document management, search, traceability, timeline reconstruction, and background processing.

The project also demonstrates the realities of software evolution. The repository now contains more functionality than the original minimal specification, including OCR-oriented extraction, image and spreadsheet support, translation, decision/risk/action registers, and startup synchronization from a local meeting corpus. At the same time, the repository still contains mismatches between code, settings, requirements, and README documentation. Recognizing and documenting those mismatches was itself an important part of this internship exercise.

Overall, the internship provided meaningful exposure to full-stack development, local AI integration, technical documentation, and applied knowledge-management problems in a semiconductor development context.

References

1. FastAPI Documentation
2. SQLAlchemy Documentation
3. SQLite Documentation
4. Ollama Local API Documentation
5. Python `pdfplumber` Documentation
6. Python `python-docx` Documentation
7. UX4G Government Template Resources present in the repository
8. Repository files reviewed under `app/`, `dataset/`, `meetings/`, and `report/`